



Instituto Politécnico Nacional
Unidad Profesional Interdisciplinaria de Ingeniería y
Ciencias Sociales y Administrativas



REPORTE DEL TERCER PARCIAL

Equipo

Núñez Velazco Saúl | NL = 33
Ríos Solorio Samanta | NL = 35
Sánchez Romero Diego | NL = 37

Profesor:

Yventz Entzana Garduño

Unidad de Aprendizaje:

ABSTRACCIÓN Y USO DE DATOS

Secuencia:

3CM30

Fecha de entrega:

18 de Junio del 2026

LINK:
https://github.com/SaulINV/ABSTRACCION_USO_DATOS

Este reporte se hizo con la finalidad de poder documentar el análisis, los errores que tuvimos al momento de programar las prácticas, así como la identificación y corrección de estos mismos, para conocer por qué la falla de estos y entender para qué mismo sirve cada archivo generado en los proyectos. Estos programas fueron desarrollados en el lenguaje C++, que implementa una clase denominada “Alumno”. La práctica realizada el martes 24 de febrero del 2026 constó de tres archivos fuente: main.cpp, Alumno.h y Alumno.cpp, para posteriormente compilarlos y visualizar su correcta ejecución, así cómo saber el contenido y conocer (o al menos poder conocer) más acerca de la sintaxis vista en clase, el proceso y las salidas del código, como, una de ellas, el saber el tamaño de una variable o el código generado. Este documento detalla cada uno de los problemas encontrados, las soluciones aplicadas y los resultados obtenidos después de las correcciones.

El desarrollo de esta práctica tuvo como objetivo principal la aplicación de los pilares de la Programación Orientada a Objetos (POO), específicamente la abstracción y el encapsulamiento. El programa original implementa una clase llamada Alumno, diseñada como una plantilla para moldear la información básica de un estudiante del mundo real, agrupando atributos esenciales como su nombre, número de boleta y promedio académico.

Dentro de la arquitectura de esta clase, se incluyeron componentes fundamentales para su ciclo de vida:

- **Un constructor:** Encargado de inicializar y preparar los datos del objeto en el momento en que es creado en la memoria.
- **Un destructor:** Implementado como una buena práctica de programación en C++ para garantizar la correcta liberación de los recursos y la memoria una vez que el objeto termina su ciclo de vida.
- **Polimorfismo (Sobrecarga):** Se desarrollaron dos métodos sobrecargados llamados msg. Ambos comparten el mismo identificador, pero cuentan con firmas distintas (diferentes parámetros). Esto permite que el programa sea flexible y que el compilador decida en tiempo de ejecución qué versión del método invocar dependiendo de si se le mandan datos o no, mostrando los mensajes correspondientes en pantalla.

Por su parte, el archivo principal (main.cpp) fue diseñado para actuar como el entorno de pruebas de nuestra clase. Su propósito es demostrar el funcionamiento práctico de los objetos instanciados y, muy importante, analizar el impacto en la memoria del sistema. Mediante el uso de la función sizeof(), el programa calcula y expone los tamaños en bytes que ocupan en la memoria tanto una estructura tradicional (struct AlumnoS) como nuestro objeto Alumno, permitiéndonos comparar cómo el lenguaje C++ gestiona el espacio al empaquetar datos complejos.

Práctica 7. Creación de tipos de datos en PE y POO

Objetivo

Desarrollar nuevos tipos de datos utilizando programación estructurada y programación orientada a objetos, además de identificar el tamaño en memoria de los tipos de datos básicos y de los tipos creados por el usuario, almacenando la información en distintos formatos de archivo.

Descripción

En esta práctica se implementaron estructuras (struct) para representar un automóvil y una persona mediante programación estructurada, así como las clases AutoPOO y PersonaPOO empleando programación orientada a objetos. Se trabajó con atributos, constructores, destructores y métodos para mostrar información.

También se utilizó el operador sizeof para conocer el tamaño en memoria de los tipos de datos básicos y de las estructuras y clases creadas.

Finalmente, se implementó la generación de archivos en formato TXT, JSON, XML y HTML, permitiendo representar la misma información mediante distintas estructuras de almacenamiento.

Archivos que contiene

main.cpp

Contiene el programa principal y el menú que permite visualizar la información o generar los diferentes tipos de archivos.

```
#include <iostream>
#include <cstring>
#include "PeYPoo.h"

using namespace std;

int main() {
    AutoPE op1;
    op1.precio = 152456532;
    op1.anio = 2025;

    PersonaPE op2;
    strcpy(op2.nombre, "Diego");
    strcpy(op2.ap, "Sanchez");
    strcpy(op2.am, "Romero");
    op2.genero = 'H';
    op2.edad = 19;

    AutoPOO op3(4243456, 2024);
    PersonaPOO op4("Samanta", "Rios", "Solorios", 'M', 19);

    int opcion;

    do {
        cout << "\nMENU \n";
        cout << "1. Mostrar datos en pantalla\n";
        cout << "2. Generar TXT\n";
        cout << "3. Generar JSON\n";
        cout << "4. Generar XML\n";
        cout << "5. Generar HTML\n";
        cout << "6. Generar TODOS\n";
        cout << "0. Salir\n";
        cout << "Opcion: ";
        cin >> opcion;

        switch (opcion) {
            case 1:
                cout << "\nPROGRAMACION ESTRUCTURADA\n";
                cout << "Auto Precio: $" << op1.precio << ", Anio: " << op1.anio << "\n";
                cout << "Persona Nombre: " << op2.nombre << " " << op2.ap << " " << op2.am
                    << ", Genero: " << op2.genero << ", Edad: " << op2.edad << "\n\n";

                cout << "PROGRAMACION ORIENTADA A OBJETOS\n";
                op3.mostrar();
            
```

```

        op4.mostrar();

        cout << "\nTAMANOS EN MEMORIA\n";
        cout << "Tamanio char: " << sizeof(char) << " bytes\n";
        cout << "Tamanio short: " << sizeof(short) << " bytes\n";
        cout << "Tamanio int: " << sizeof(int) << " bytes\n";
        cout << "Tamanio long: " << sizeof(long) << " bytes\n";
        cout << "Tamanio float: " << sizeof(float) << " bytes\n";
        cout << "Tamanio double: " << sizeof(double) << " bytes\n";

        cout << "Tamanio AutoPE: " << sizeof(AutoPE) << " bytes\n";
        cout << "Tamanio PersonaPE: " << sizeof(PersonaPE) << " bytes\n";
        cout << "Tamanio AutoPOO: " << sizeof(AutoPOO) << " bytes\n";
        cout << "Tamanio PersonaPOO: " << sizeof(PersonaPOO) << " bytes\n";
        break;

    case 2:
        generarTXT(op1, op2, op3, op4);
        break;

    case 3:
        generarJSON(op1, op2, op3, op4);
        break;

    case 4:
        generarXML(op1, op2, op3, op4);
        break;

    case 5:
        generarHTML(op1, op2, op3, op4);
        break;

    case 6:
        generarTXT(op1, op2, op3, op4);
        generarJSON(op1, op2, op3, op4);
        generarXML(op1, op2, op3, op4);
        generarHTML(op1, op2, op3, op4);
        cout << "\nTodos los archivos fueron generados.\n";
        break;

    case 0:
        cout << " ";
        break;

    default:
        cout << "Opcion invalida.\n";
    }
}

} while (opcion != 0);

return 0;
}

```

PeYPoo.cpp

Implementa los constructores, destructores, métodos de acceso y las funciones encargadas de generar los archivos TXT, JSON, XML y HTML.

```
#include "PeYPoo.h"
#include <fstream>

AutoP00::AutoP00(int p, int a) {
    precio = p;
    anio = a;
}

AutoP00::~AutoP00() {
}

int AutoP00::getPrecio() {
    return precio;
}

int AutoP00::getAnio() {
    return anio;
}

void AutoP00::mostrar() {
    cout << "Auto P00: $" << precio << ", Anio: " << anio << "\n";
}

PersonaP00::PersonaP00(const char n[], const char p[], const char m[], char g, int e) {
    strcpy(nombre, n);
    strcpy(ap, p);
    strcpy(am, m);
    genero = g;
    edad = e;
}

PersonaP00::~PersonaP00() {
}

const char* PersonaP00::getNombre() {
    return nombre;
}

const char* PersonaP00::getAp() {
    return ap;
}

const char* PersonaP00::getAm() {
    return am;
}
```

```

const char* PersonaPOO::getAm() {
    return am;
}

char PersonaPOO::getGenero() {
    return genero;
}

int PersonaPOO::getEdad() {
    return edad;
}

void PersonaPOO::mostrar() {
    cout << "Persona Nombre: " << nombre << " " << ap << " " << am
    << " ", Genero: " << genero << " ", Edad: " << edad << "\n";
}

void generarTXT(AutoPE autoPE, PersonaPE personaPE, AutoPOO autoPOO, PersonaPOO personaPOO) {
    ofstream archivo("practica7.txt");

    archivo << "PRACTICA 7 - CREACION DE DATO EN PE Y POO\n\n";

    archivo << "PROGRAMACION ESTRUCTURADA\n";
    archivo << "AutoPE: Precio $" << autoPE.precio << " ", Anio " << autoPE.anio << "\n";
    archivo << "PersonaPE: " << personaPE.nombre << " " << personaPE.ap << " " << personaPE.am
    << " ", Genero " << personaPE.genero << " ", Edad " << personaPE.edad << "\n\n";

    archivo << "PROGRAMACION ORIENTADA A OBJETOS\n";
    archivo << "AutoPOO: Precio $" << autoPOO.getPrecio() << " ", Anio " << autoPOO.getAnio() << "\n";
    archivo << "PersonaPOO: " << personaPOO.getNombre() << " " << personaPOO.getAp() << " " << personaPOO.getAm()
    << " ", Genero " << personaPOO.getGenero() << " ", Edad " << personaPOO.getEdad() << "\n\n";

    archivo << "PRACTICA 7 BIS - TAMAÑOS EN MEMORIA\n";
    archivo << "char: " << sizeof(char) << " bytes\n";
    archivo << "short: " << sizeof(short) << " bytes\n";
    archivo << "int: " << sizeof(int) << " bytes\n";
    archivo << "long: " << sizeof(long) << " bytes\n";
    archivo << "float: " << sizeof(float) << " bytes\n";
    archivo << "double: " << sizeof(double) << " bytes\n";
    archivo << "AutoPE: " << sizeof(AutoPE) << " bytes\n";
    archivo << "PersonaPE: " << sizeof(PersonaPE) << " bytes\n";
    archivo << "AutoPOO: " << sizeof(AutoPOO) << " bytes\n";
    archivo << "PersonaPOO: " << sizeof(PersonaPOO) << " bytes\n";

    archivo.close();

    cout << "Archivo practica7.txt generado.\n";
}

void generarJSON(AutoPE autoPE, PersonaPE personaPE, AutoPOO autoPOO, PersonaPOO personaPOO) {
    ofstream archivo("practica7.json");

    archivo << "{\n";
    archivo << "  \"Practica\": \"Practica 7 - Creacion de dato PE y POO\", \n";
    archivo << "  \"ProgramacionEstructurada\": {\n";
    archivo << "    \"AutoPE\": {\n";
    archivo << "      \"precio\": " << autoPE.precio << " , \n";
    archivo << "      \"anio\": " << autoPE.anio << "\n";
    archivo << "    }, \n";
    archivo << "    \"PersonaPE\": {\n";

```

```

archivo << " \nombre\": \"\" << personaPE.nombre << "\",\n";
archivo << " \apellidoPaterno\": \"\" << personaPE.ap << "\",\n";
archivo << " \apellidoMaterno\": \"\" << personaPE.am << "\",\n";
archivo << " \genero\": \"\" << personaPE.genero << "\",\n";
archivo << " \edad\": \"\" << personaPE.edad << "\",\n";
archivo << " }\n";
archivo << " },\n";
archivo << " \ProgramacionOrientadaObjetos\": {\n";
archivo << " \AutoP00\": {\n";
archivo << " \precio\": \"\" << autoP00.getPrecio() << "\",\n";
archivo << " \anio\": \"\" << autoP00.getAnio() << "\",\n";
archivo << " },\n";
archivo << " \PersonaP00\": {\n";
archivo << " \nombre\": \"\" << personaP00.getNombre() << "\",\n";
archivo << " \apellidoPaterno\": \"\" << personaP00.getAp() << "\",\n";
archivo << " \apellidoMaterno\": \"\" << personaP00.getAm() << "\",\n";
archivo << " \genero\": \"\" << personaP00.getGenero() << "\",\n";
archivo << " \edad\": \"\" << personaP00.getEdad() << "\",\n";
archivo << " }\n";
archivo << " },\n";
archivo << " \Tamanos\": {\n";
archivo << " \char\": \"\" << sizeof(char) << "\",\n";
archivo << " \short\": \"\" << sizeof(short) << "\",\n";
archivo << " \int\": \"\" << sizeof(int) << "\",\n";
archivo << " \long\": \"\" << sizeof(long) << "\",\n";
archivo << " \float\": \"\" << sizeof(float) << "\",\n";
archivo << " \double\": \"\" << sizeof(double) << "\",\n";
archivo << " \AutoPE\": \"\" << sizeof(AutoPE) << "\",\n";
archivo << " \PersonaPE\": \"\" << sizeof(PersonaPE) << "\",\n";
archivo << " \AutoP00\": \"\" << sizeof(AutoP00) << "\",\n";
archivo << " \PersonaP00\": \"\" << sizeof(PersonaP00) << "\",\n";
archivo << " }\n";
archivo << " }\n";

archivo.close();

cout << "Archivo practica7.json generado.\n";
}

void generarXML(AutoPE autoPE, PersonaPE personaPE, AutoP00 autoP00, PersonaP00 personaP00) {
ofstream archivo("practica7.xml");

archivo << "<?xml version='1.0' encoding='UTF-8'>\n";
archivo << "<Practica7>\n";

archivo << " <ProgramacionEstructurada>\n";
archivo << " <AutoPE>\n";
archivo << " <precio>\" << autoPE.precio << "</precio>\n";
archivo << " <anio>\" << autoPE.anio << "</anio>\n";
archivo << " </AutoPE>\n";
archivo << " <PersonaPE>\n";
archivo << " <nombre>\" << personaPE.nombre << "</nombre>\n";
archivo << " <apellidoPaterno>\" << personaPE.ap << "</apellidoPaterno>\n";
archivo << " <apellidoMaterno>\" << personaPE.am << "</apellidoMaterno>\n";
archivo << " <genero>\" << personaPE.genero << "</genero>\n";
archivo << " <edad>\" << personaPE.edad << "</edad>\n";
archivo << " </PersonaPE>\n";
archivo << " </ProgramacionEstructurada>\n";

```

```

archivo << " <ProgramacionOrientadaObjetos>\n";
archivo << " <AutoP00>\n";
archivo << " <precio>" << autoP00.getPrecio() << "</precio>\n";
archivo << " <anio>" << autoP00.getAnio() << "</anio>\n";
archivo << " </AutoP00>\n";
archivo << " <PersonaP00>\n";
archivo << " <nombre>" << personaP00.getNombre() << "</nombre>\n";
archivo << " <apellidoPaterno>" << personaP00.getAp() << "</apellidoPaterno>\n";
archivo << " <apellidoMaterno>" << personaP00.getAm() << "</apellidoMaterno>\n";
archivo << " <genero>" << personaP00.getGenero() << "</genero>\n";
archivo << " <edad>" << personaP00.getEdad() << "</edad>\n";
archivo << " </PersonaP00>\n";
archivo << " </ProgramacionOrientadaObjetos>\n";

archivo << " <Tamanos>\n";
archivo << " <char>" << sizeof(char) << "</char>\n";
archivo << " <short>" << sizeof(short) << "</short>\n";
archivo << " <int>" << sizeof(int) << "</int>\n";
archivo << " <long>" << sizeof(long) << "</long>\n";
archivo << " <float>" << sizeof(float) << "</float>\n";
archivo << " <double>" << sizeof(double) << "</double>\n";
archivo << " <AutoPE>" << sizeof(AutoPE) << "</AutoPE>\n";
archivo << " <PersonaPE>" << sizeof(PersonaPE) << "</PersonaPE>\n";
archivo << " <AutoP00>" << sizeof(AutoP00) << "</AutoP00>\n";
archivo << " <PersonaP00>" << sizeof(PersonaP00) << "</PersonaP00>\n";
archivo << " </Tamanos>\n";

archivo << "</Practica7>\n";

archivo.close();

cout << "Archivo practica7.xml generado.\n";

void generarHTML(AutoPE autoPE, PersonaPE personaPE, AutoP00 autoP00, PersonaP00 personaP00) {
    ofstream archivo("practica7.html");

    archivo << "<!DOCTYPE html>\n";
    archivo << "<html lang=\"es\">\n";
    archivo << "<head>\n";
    archivo << " <meta charset=\"UTF-8\">\n";
    archivo << " <title>Practica 7</title>\n";
    archivo << "</head>\n";
    archivo << "<body>\n";

    archivo << " <h1>Practica 7 - Creacion de dato PE y P00</h1>\n";

    archivo << " <h2>Programacion Estructurada</h2>\n";
    archivo << " <p>AutoPE: Precio $" << autoPE.precio << ", Anio " << autoPE.anio << "</p>\n";
    archivo << " <p>PersonaPE: " << personaPE.nombre << " " << personaPE.ap << " " << personaPE.am
    << ", Genero " << personaPE.genero << ", Edad " << personaPE.edad << "</p>\n";

    archivo << " <h2>Programacion Orientada a Objetos</h2>\n";
    archivo << " <p>AutoP00: Precio $" << autoP00.getPrecio() << ", Anio " << autoP00.getAnio() << "</p>\n";
    archivo << " <p>PersonaP00: " << personaP00.getNombre() << " " << personaP00.getAp() << " " << personaP00.getAm()
    << ", Genero " << personaP00.getGenero() << ", Edad " << personaP00.getEdad() << "</p>\n";

    archivo << " <h2>Tamanos en memoria</h2>\n";
    archivo << " <ul>\n";
    archivo << " <li>char: " << sizeof(char) << " bytes</li>\n";

    archivo << " <li>short: " << sizeof(short) << " bytes</li>\n";
    archivo << " <li>int: " << sizeof(int) << " bytes</li>\n";
    archivo << " <li>long: " << sizeof(long) << " bytes</li>\n";
    archivo << " <li>float: " << sizeof(float) << " bytes</li>\n";
    archivo << " <li>double: " << sizeof(double) << " bytes</li>\n";
    archivo << " <li>AutoPE: " << sizeof(AutoPE) << " bytes</li>\n";
    archivo << " <li>PersonaPE: " << sizeof(PersonaPE) << " bytes</li>\n";
    archivo << " <li>AutoP00: " << sizeof(AutoP00) << " bytes</li>\n";
    archivo << " <li>PersonaP00: " << sizeof(PersonaP00) << " bytes</li>\n";
    archivo << " </ul>\n";

    archivo << "</body>\n";
    archivo << "</html>\n";

    archivo.close();

    cout << "Archivo practica7.html generado.\n";
}

```


PeYPoo.h

Contiene la definición de las estructuras y clases utilizadas, así como los prototipos de las funciones encargadas de generar los archivos.

```
// PeYPoo.h
#ifndef PEYPOO_H
#define PEYPOO_H

#include <iostream>
#include <cstring>
using namespace std;

// PROGRAMACION ESTRUCTURADA
struct AutoPE {
    int precio;
    int anio;
};

struct PersonaPE {
    char nombre[20];
    char ap[20];
    char am[20];
    char genero;
    int edad;
};

// PROGRAMACION ORIENTADA A OBJETOS
class AutoPOO {
private:
    int precio;
    int anio;
public:
    AutoPOO(int p, int a);
    virtual ~AutoPOO();

    int getPrecio();
    int getAnio();
    void mostrar();
};

class PersonaPOO {
private:
    char nombre[20];
    char ap[20];
    char am[20];
    char genero;
    int edad;
public:
    PersonaPOO(const char n[], const char p[], const char m[], char g, int e);
    virtual ~PersonaPOO();

    const char* getNombre();
    const char* getAp();
    const char* getAm();
    char getGenero();
    int getEdad();

    void mostrar();
};

// FUNCIONES PARA GENERAR ARCHIVOS
void generarTXT(AutoPE autoPE, PersonaPE personaPE, AutoPOO autoPOO, PersonaPOO personaPOO);
void generarJSON(AutoPE autoPE, PersonaPE personaPE, AutoPOO autoPOO, PersonaPOO personaPOO);
void generarXML(AutoPE autoPE, PersonaPE personaPE, AutoPOO autoPOO, PersonaPOO personaPOO);

void generarHTML(AutoPE autoPE, PersonaPE personaPE, AutoPOO autoPOO, PersonaPOO personaPOO);

#endif
```

Práctica 8. Operaciones estadísticas con variables independientes

Objetivo

Desarrollar un programa que permita capturar cinco números y obtener la suma, promedio, media, máximo y mínimo, almacenando los resultados en diferentes formatos de archivo.

Descripción

Se implementó una clase llamada Numeros, la cual almacena cinco variables independientes. Mediante funciones miembro se realizaron las operaciones de suma, promedio, media, valor máximo y valor mínimo.

Se incorporó un menú para visualizar los resultados y generar archivos en formato TXT, JSON, XML y HTML, permitiendo observar diferentes formas de representación de la información.

Archivos que contiene

mainNumeros.cpp

Contiene el programa principal, la captura de datos y el menú para mostrar resultados o generar archivos.

```
#include <iostream>
#include "Numeros.h"

using namespace std;

int main() {

    float a, b, c, d, e;

    cout << "Numero 1: ";
    cin >> a;

    cout << "Numero 2: ";
    cin >> b;

    cout << "Numero 3: ";
    cin >> c;

    cout << "Numero 4: ";
    cin >> d;

    cout << "Numero 5: ";
    cin >> e;

    Numeros op(a, b, c, d, e);

    int opcion;

    do {

        cout << "\n MENU PRACTICA 8 \n";
        cout << "1. Mostrar resultados\n";
        cout << "2. Generar TXT\n";
        cout << "3. Generar JSON\n";
        cout << "4. Generar XML\n";
        cout << "5. Generar HTML\n";
        cout << "6. Generar TODOS\n";
        cout << "0. Salir\n";
        cout << "Opcion: ";

        cin >> opcion;

        switch (opcion) {

            case 1:

                cout << "\nRESULTADOS\n";
                cout << "Suma: " << op.suma() << "\n";
                cout << "Promedio: " << op.promedio() << "\n";
                cout << "Media: " << op.media() << "\n";
                cout << "Maximo: " << op.maximo() << "\n";
                cout << "Minimo: " << op.minimo() << "\n";

                break;

            case 2:

                generarTXT(op);
                break;

            case 3:

                generarJSON(op);
                break;

            case 4:

                generarXML(op);
                break;

            case 5:

                generarHTML(op);
                break;

            case 6:

                generarTXT(op);
```

```

        case 6:
            generarTXT(op);
            generarJSON(op);
            generarXML(op);
            generarHTML(op);

            cout << "\nTodos los archivos fueron generados.\n";

            break;

        case 0:
            cout << "Saliendo...\n";
            break;

        default:
            cout << "Opcion invalida.\n";
    }
} while (opcion != 0);

return 0;

```

Numeros.h

Contiene la declaración de la clase Numeros, sus atributos y los prototipos de las funciones.

```

#ifndef NUMEROS_H
#define NUMEROS_H

class Numeros {
private:
    float n1, n2, n3, n4, n5;

public:
    Numeros(float a, float b, float c, float d, float e);
    ~Numeros();

    float suma();
    float promedio();
    float media();
    float maximo();
    float minimo();

    float getN1();
    float getN2();
    float getN3();
    float getN4();
    float getN5();
};

void generarTXT(Numeros op);
void generarJSON(Numeros op);
void generarXML(Numeros op);
void generarHTML(Numeros op);

#endif

```

Numeros.cpp

Implementa los métodos encargados de realizar las operaciones matemáticas y las funciones para generar los archivos TXT, JSON, XML y HTML.

```

#include "Numeros.h"
#include <iostream>
#include <fstream>

using namespace std;

Numeros::Numeros(float a, float b, float c, float d, float e) {
    n1 = a;
    n2 = b;
    n3 = c;
    n4 = d;
    n5 = e;
}

Numeros::~Numeros() {
}

float Numeros::suma() {
    return n1 + n2 + n3 + n4 + n5;
}

float Numeros::promedio() {
    return suma() / 5.0;
}

float Numeros::media() {
    return suma() / 5.0;
}

float Numeros::maximo() {
    float m = n1;

    if (n2 > m) m = n2;
    if (n3 > m) m = n3;
    if (n4 > m) m = n4;
    if (n5 > m) m = n5;

    return m;
}

float Numeros::minimo() {
    float m = n1;

    if (n2 < m) m = n2;
    if (n3 < m) m = n3;
    if (n4 < m) m = n4;
    if (n5 < m) m = n5;

    return m;
}

float Numeros::getN1() { return n1; }
float Numeros::getN2() { return n2; }
float Numeros::getN3() { return n3; }
float Numeros::getN4() { return n4; }
float Numeros::getN5() { return n5; }

// TXT
void generarTXT(Numeros op) {

    ofstream archivo("practica8.txt");

    archivo << "PRACTICA 8 - NUMEROS\n\n";

    archivo << "Numero 1: " << op.getN1() << "\n";
    archivo << "Numero 2: " << op.getN2() << "\n";
    archivo << "Numero 3: " << op.getN3() << "\n";
    archivo << "Numero 4: " << op.getN4() << "\n";
    archivo << "Numero 5: " << op.getN5() << "\n\n";

    archivo << "Suma: " << op.suma() << "\n";
    archivo << "Promedio: " << op.promedio() << "\n";
    archivo << "Media: " << op.media() << "\n";
    archivo << "Maximo: " << op.maximo() << "\n";
}

```

```

    archivo << "Minimo: " << op.minimo() << "\n";

    archivo.close();

    cout << "Archivo practica8.txt generado.\n";
}

// JSON
void generarJSON(Numeros op) {

    ofstream archivo("practica8.json");

    archivo << "{\n";
    archivo << "  \"Practica\": \"Practica 8\", \n";
    archivo << "  \"Numeros\": [\n";
    archivo << "    << op.getN1() << ", \n";
    archivo << "    << op.getN2() << ", \n";
    archivo << "    << op.getN3() << ", \n";
    archivo << "    << op.getN4() << ", \n";
    archivo << "    << op.getN5() << "\n";
    archivo << "  ], \n";

    archivo << "  \"Resultados\": {\n";
    archivo << "    \"suma\": " << op.suma() << ", \n";
    archivo << "    \"promedio\": " << op.promedio() << ", \n";
    archivo << "    \"media\": " << op.media() << ", \n";
    archivo << "    \"maximo\": " << op.maximo() << ", \n";
    archivo << "    \"minimo\": " << op.minimo() << "\n";
    archivo << "  } \n";
    archivo << "}\n";

    archivo.close();

    cout << "Archivo practica8.json generado.\n";
}

// XML
void generarXML(Numeros op) {

    ofstream archivo("practica8.xml");

    archivo << "<?xml version='1.0' encoding='UTF-8'?'>\n";
    archivo << "<Practica8>\n";

    archivo << "  <Numeros>\n";
    archivo << "    <Numero1>" << op.getN1() << "</Numero1>\n";
    archivo << "    <Numero2>" << op.getN2() << "</Numero2>\n";
    archivo << "    <Numero3>" << op.getN3() << "</Numero3>\n";
    archivo << "    <Numero4>" << op.getN4() << "</Numero4>\n";
    archivo << "    <Numero5>" << op.getN5() << "</Numero5>\n";
    archivo << "  </Numeros>\n";

    archivo << "  <Resultados>\n";
    archivo << "    <Suma>" << op.suma() << "</Suma>\n";
    archivo << "    <Promedio>" << op.promedio() << "</Promedio>\n";
    archivo << "    <Media>" << op.media() << "</Media>\n";
    archivo << "    <Maximo>" << op.maximo() << "</Maximo>\n";
    archivo << "    <Minimo>" << op.minimo() << "</Minimo>\n";
    archivo << "  </Resultados>\n";

    archivo << "</Practica8>\n";

    archivo.close();

    cout << "Archivo practica8.xml generado.\n";
}

// HTML
void generarHTML(Numeros op) {

    ofstream archivo("practica8.html");

    archivo << "<!DOCTYPE html>\n";

```

```

archivo << "<!DOCTYPE html>\n";
archivo << "<html lang=\"es\">\n";
archivo << "<head>\n";
archivo << "  <meta charset=\"UTF-8\">\n";
archivo << "  <title>Practica 8</title>\n";
archivo << "</head>\n";
archivo << "<body>\n";

archivo << "<h1>Practica 8 - Numeros</h1>\n";

archivo << "<h2>Numeros ingresados</h2>\n";
archivo << "<ul>\n";
archivo << "<li>" << op.getN1() << "</li>\n";
archivo << "<li>" << op.getN2() << "</li>\n";
archivo << "<li>" << op.getN3() << "</li>\n";
archivo << "<li>" << op.getN4() << "</li>\n";
archivo << "<li>" << op.getN5() << "</li>\n";
archivo << "</ul>\n";

archivo << "<h2>Resultados</h2>\n";
archivo << "<p>Suma: " << op.suma() << "</p>\n";
archivo << "<p>Promedio: " << op.promedio() << "</p>\n";
archivo << "<p>Media: " << op.media() << "</p>\n";
archivo << "<p>Maximo: " << op.maximo() << "</p>\n";
archivo << "<p>Minimo: " << op.minimo() << "</p>\n";

archivo << "</body>\n";
archivo << "</html>\n";

archivo.close();

cout << "Archivo practica8.html generado.\n";
}

```

Práctica 9. Operaciones estadísticas utilizando arreglos

Objetivo

Implementar un programa que permita almacenar cinco valores en un arreglo y obtener la suma, promedio, media, máximo y mínimo, generando los resultados en distintos formatos de archivo.

Descripción

En esta práctica se empleó un arreglo para almacenar los cinco números ingresados por el usuario. A través de ciclos se realizaron las operaciones de suma, promedio, media, máximo y mínimo.

La información obtenida fue almacenada en archivos TXT, JSON, XML y HTML, permitiendo analizar diferentes formas de organización y representación de los datos.

Archivos que contiene

mainNumerosArreglo.cpp

Contiene la captura de datos y el menú principal del programa.

```

1  #include <iostream>
2  #include "NumerosArreglo.h"
3
4  using namespace std;
5
6  int main() {
7      float valores[5];
8
9      for (int i = 0; i < 5; i++) {
10         cout << "Ingresa el numero " << i + 1 << ": ";
11         cin >> valores[i];
12     }
13
14     Numeros op(valores);
15
16     int opcion;
17
18     do {
19         cout << "\n MENU PRACTICA 9 \n";
20         cout << "1. Mostrar resultados\n";
21         cout << "2. Generar TXT\n";
22         cout << "3. Generar JSON\n";
23         cout << "4. Generar XML\n";
24         cout << "5. Generar HTML\n";
25         cout << "6. Generar TODOS\n";
26         cout << "0. Salir\n";
27         cout << "Opcion: ";
28
29         cin >> opcion;
30
31         switch (opcion) {
32             case 1:
33                 cout << "\nRESULTADOS\n";
34                 cout << "Suma: " << op.suma() << "\n";
35                 cout << "Promedio: " << op.promedio() << "\n";
36                 cout << "Media: " << op.media() << "\n";
37                 cout << "Maximo: " << op.maximo() << "\n";
38                 cout << "Minimo: " << op.minimo() << "\n";
39                 break;
40
41             case 2:
42                 generarTXT(op);
43                 break;
44
45             case 3:
46                 generarJSON(op);
47                 break;
48
49             case 4:
50                 generarXML(op);
51                 break;
52
53             case 5:
54                 generarHTML(op);
55                 break;
56
57             case 6:
58                 generarTXT(op);
59                 generarJSON(op);
60                 generarXML(op);
61                 generarHTML(op);
62                 cout << "\nTodos los archivos fueron generados.\n";
63                 break;
64
65             case 0:
66                 cout << "\n";
67                 break;
68
69             default:
70                 cout << "Opcion invalida.\n";
71         }
72     } while (opcion != 0);
73
74

```


NumerosArreglo.cpp

Implementa los métodos necesarios para recorrer el arreglo, realizar las operaciones estadísticas y generar los archivos TXT, JSON, XML y HTML.

```
#include "NumerosArreglo.h"
#include <iostream>
#include <fstream>

using namespace std;

Numeros::Numeros(float v[]) {
    for (int i = 0; i < 5; i++) {
        datos[i] = v[i];
    }
}

Numeros::~Numeros() {
}

float Numeros::suma() {
    float s = 0;

    for (int i = 0; i < 5; i++) {
        s += datos[i];
    }

    return s;
}

float Numeros::promedio() {
    return suma() / 5.0;
}

float Numeros::media() {
    return suma() / 5.0;
}

float Numeros::maximo() {
    float m = datos[0];

    for (int i = 1; i < 5; i++) {
        if (datos[i] > m) {
            m = datos[i];
        }
    }

    return m;
}

float Numeros::minimo() {
    float m = datos[0];

    for (int i = 1; i < 5; i++) {
        if (datos[i] < m) {
            m = datos[i];
        }
    }

    return m;
}

float Numeros::getDato(int i) {
    return datos[i];
}

void generarTXT(Numeros op) {
    ofstream archivo("practica9.txt");

    archivo << "PRACTICA 9 - ARREGLOS\n\n";

    for (int i = 0; i < 5; i++) {
        archivo << "Numero " << i + 1 << ": " << op.getDato(i) << "\n";
    }

    archivo << "\nRESULTADOS\n";
    archivo << "Suma: " << op.suma() << "\n";
    archivo << "Promedio: " << op.promedio() << "\n";
    archivo << "Media: " << op.media() << "\n";
}
```

```

archivo << "Maximo: " << op.maximo() << "\n";
archivo << "Minimo: " << op.minimo() << "\n";

archivo.close();

cout << "Archivo practica9.txt generado.\n";
}

void generarJSON(Numeros op) {
    ofstream archivo("practica9.json");

    archivo << "{\n";
    archivo << "  \"Practica\": \"Practica 9 - Arreglos\",\n";
    archivo << "  \"Numeros\": [\n";

    for (int i = 0; i < 5; i++) {
        archivo << "    " << op.getDatos(i);

        if (i < 4) {
            archivo << ",";
        }

        archivo << "\n";
    }

    archivo << "  ],\n";
    archivo << "  \"Resultados\": {\n";
    archivo << "    \"suma\": " << op.suma() << ",\n";
    archivo << "    \"promedio\": " << op.promedio() << ",\n";
    archivo << "    \"media\": " << op.media() << ",\n";
    archivo << "    \"maximo\": " << op.maximo() << ",\n";
    archivo << "    \"minimo\": " << op.minimo() << "\n";
    archivo << "  }\n";
    archivo << "}\n";

    archivo.close();

    cout << "Archivo practica9.json generado.\n";
}

void generarXML(Numeros op) {
    ofstream archivo("practica9.xml");

    archivo << "<?xml version='1.0' encoding='UTF-8'>\n";
    archivo << "<Practica9>\n";

    archivo << "  <Numeros>\n";

    for (int i = 0; i < 5; i++) {
        archivo << "    <Numero>" << op.getDatos(i) << "</Numero>\n";
    }

    archivo << "  </Numeros>\n";

    archivo << "  <Resultados>\n";
    archivo << "    <Suma>" << op.suma() << "</Suma>\n";
    archivo << "    <Promedio>" << op.promedio() << "</Promedio>\n";
    archivo << "    <Media>" << op.media() << "</Media>\n";
    archivo << "    <Maximo>" << op.maximo() << "</Maximo>\n";
    archivo << "    <Minimo>" << op.minimo() << "</Minimo>\n";
    archivo << "  </Resultados>\n";

    archivo << "</Practica9>\n";

    archivo.close();

    cout << "Archivo practica9.xml generado.\n";
}

void generarHTML(Numeros op) {
    ofstream archivo("practica9.html");

    archivo << "<!DOCTYPE html>\n";

```

```

archivo << "<!DOCTYPE html>\n";
archivo << "<html lang='es'\>\n";
archivo << "<head>\n";
archivo << "    <meta charset='UTF-8'\>\n";
archivo << "    <title>Practica 9</title>\n";
archivo << "</head>\n";
archivo << "<body>\n";

archivo << "    <h1>Practica 9 - Arreglos</h1>\n";

archivo << "    <h2>Numeros ingresados</h2>\n";
archivo << "    <ul>\n";

for (int i = 0; i < 5; i++) {
    |   archivo << "        <li>Numero " << i + 1 << ": " << op.getDato(i) << "</li>\n";
}

archivo << "    </ul>\n";

archivo << "    <h2>Resultados</h2>\n";
archivo << "    <p>Suma: " << op.suma() << "</p>\n";
archivo << "    <p>Promedio: " << op.promedio() << "</p>\n";
archivo << "    <p>Media: " << op.media() << "</p>\n";
archivo << "    <p>Maximo: " << op.maximo() << "</p>\n";
archivo << "    <p>Minimo: " << op.minimo() << "</p>\n";

archivo << "</body>\n";
archivo << "</html>\n";

archivo.close();

cout << "Archivo practica9.html generado.\n";
}

```

NumerosArreglo.h

Contiene la definición de la clase Numeros, que almacena los datos mediante un arreglo de cinco posiciones, así como los prototipos de las funciones.

```

#ifndef NUMEROSARREGLO_H
#define NUMEROSARREGLO_H

class Numeros {
private:
    float datos[5];

public:
    Numeros(float v[]);
    ~Numeros();

    float suma();
    float promedio();
    float media();
    float maximo();
    float minimo();

    float getDato(int i);
};

void generarTXT(Numeros op);
void generarJSON(Numeros op);
void generarXML(Numeros op);
void generarHTML(Numeros op);

#endif

```

Práctica 10. Operaciones estadísticas utilizando punteros al arreglo

Objetivo

Desarrollar un programa que permita acceder indirectamente a los elementos de un arreglo mediante punteros para realizar operaciones estadísticas y generar distintos tipos de archivos.

Descripción

Se implementó una clase que utiliza un arreglo y un puntero asociado para acceder a los elementos mediante aritmética de punteros. Los datos fueron procesados para obtener suma, promedio, media, máximo y mínimo.

Además, se implementó la generación de archivos en formato TXT, JSON, XML y HTML, permitiendo representar la información en diferentes estructuras y reforzando el manejo de archivos en C++.

Archivos que contiene

mainP10.cpp

Contiene el programa principal, la captura de datos y el menú para mostrar resultados o generar archivos.

```
#include <iostream>
#include "NumsPunArr.h"

using namespace std;

int main() {

    float valores[5];

    for(int i=0; i<5; i++) {
        cout << "Ingresa el numero " << i+1 << ": ";
        cin >> valores[i];
    }

    Numeros op(valores);

    int opcion;

    do {

        cout << "\n PRACTICA 10 \n";
        cout << "1. Mostrar resultados\n";
        cout << "2. Generar TXT\n";
        cout << "3. Generar JSON\n";
        cout << "4. Generar XML\n";
        cout << "5. Generar HTML\n";
        cout << "6. Generar TODOS\n";
        cout << "0. Salir\n";
        cout << "Opcion: ";
        cin >> opcion;

        switch(opcion) {

            case 1:

                cout << "\nRESULTADOS\n";

                cout << "Suma: "
                    << op.suma()
                    << "\n";

                cout << "Promedio: "
                    << op.promedio()
                    << "\n";

                cout << "Media: "
                    << op.media()
                    << "\n";

                cout << "Maximo: "
                    << op.maximo()
                    << "\n";

                cout << "Minimo: "
                    << op.minimo()
                    << "\n";

                break;

            case 2:
                generarTXT(op);
                break;

        }

    } while (opcion != 0);

}
```

```

        case 3:
            generarJSON(op);
            break;

        case 4:
            generarXML(op);
            break;

        case 5:
            generarHTML(op);
            break;

        case 6:

            generarTXT(op);
            generarJSON(op);
            generarXML(op);
            generarHTML(op);

            cout << "\nTodos los archivos fueron generados.\n";

            break;

        case 0:
            cout << "\nSaliendo...\n";
            break;

        default:
            cout << "\nOpcion invalida\n";
    }

} while(opcion != 0);

return 0;
}

```

NumsPunArr.h

Contiene la definición de la clase Numeros, sus atributos y los prototipos de las funciones encargadas de generar archivos.

```

#ifndef NUMSPUNARR_H
#define NUMSPUNARR_H

class Numeros {

private:
    float datos[5];
    float *p;

public:
    Numeros(float v[]);
    ~Numeros();

    float suma();
    float promedio();
    float media();
    float maximo();
    float minimo();

    float getDato(int i);
};

void generarTXT(Numeros op);
void generarJSON(Numeros op);
void generarXML(Numeros op);
void generarHTML(Numeros op);

#endif

```

NumsPunArr.cpp

Implementa el acceso a los datos mediante punteros y la aritmética de direcciones, además de las funciones encargadas de generar los archivos TXT, JSON, XML y HTML.

```

#include "NumsPunArr.h"
#include <fstream>
#include <iostream>

using namespace std;

Numeros::Numeros(float v[]) {
    p = datos;

    for(int i=0; i<5; i++) {
        *(p+i) = v[i];
    }
}

Numeros::~Numeros() {
}

float Numeros::suma() {
    float s = 0;

    for(int i=0; i<5; i++) {
        s += *(p+i);
    }

    return s;
}

float Numeros::promedio() {
    return suma()/5.0;
}

float Numeros::media() {
    return promedio();
}

float Numeros::maximo() {
    float m = *p;

    for(int i=1; i<5; i++) {
        if(*(p+i) > m)
            m = *(p+i);
    }

    return m;
}

float Numeros::minimo() {
    float m = *p;

    for(int i=1; i<5; i++) {
        if(*(p+i) < m)
            m = *(p+i);
    }

    return m;
}

```

```

float Numeros::getDato(int i) {
    return *(p+i);
}

void generarTXT(Numeros op) {

    ofstream archivo("practica10.txt");

    archivo << "PRACTICA 10 - PUNTEROS AL ARREGLO\n\n";

    for(int i=0; i<5; i++) {

        archivo << "Numero "
                << i+1
                << ": "
                << op.getDato(i)
                << "\n";
    }

    archivo << "\nSuma: " << op.suma();
    archivo << "\nPromedio: " << op.promedio();
    archivo << "\nMedia: " << op.media();
    archivo << "\nMaximo: " << op.maximo();
    archivo << "\nMinimo: " << op.minimo();

    archivo.close();

    cout << "Archivo practica10.txt generado.\n";
}

void generarJSON(Numeros op) {

    ofstream archivo("practica10.json");

    archivo << "{\n";
    archivo << "\"Numeros\":[\n";

    for(int i=0; i<5; i++) {

        archivo << op.getDato(i);

        if(i<4)
            archivo << ",";

        archivo << "\n";
    }

    archivo << "],\n";

    archivo << "\"Resultados\":{\n";
    archivo << "\"suma\": " << op.suma() << ",\n";
    archivo << "\"promedio\": " << op.promedio() << ",\n";
    archivo << "\"media\": " << op.media() << ",\n";
    archivo << "\"maximo\": " << op.maximo() << ",\n";
    archivo << "\"minimo\": " << op.minimo() << "\n";
    archivo << "}\n";
    archivo << "}";

    archivo.close();

    cout << "Archivo practica10.json generado.\n";
}

void generarXML(Numeros op) {

    ofstream archivo("practica10.xml");

    archivo << "<?xml version='1.0'>\n";
    archivo << "<Practica10>\n";

    archivo << "<Numeros>\n";

    for(int i=0; i<5; i++) {

```



```

        archivo << "<Numero>"
        << op.getDatos(i)
        << "</Numero>\n";
    }

    archivo << "</Numeros>\n";

    archivo << "<Resultados>\n";
    archivo << "<Suma>" << op.suma() << "</Suma>\n";
    archivo << "<Promedio>" << op.promedio() << "</Promedio>\n";
    archivo << "<Media>" << op.media() << "</Media>\n";
    archivo << "<Maximo>" << op.maximo() << "</Maximo>\n";
    archivo << "<Minimo>" << op.minimo() << "</Minimo>\n";
    archivo << "</Resultados>\n";

    archivo << "</Practica10>";

    archivo.close();

    cout << "Archivo practica10.xml generado.\n";
}

```

```

void generarHTML(Numeros op) {

    ofstream archivo("practica10.html");

    archivo << "<!DOCTYPE html>\n";
    archivo << "<html>\n";
    archivo << "<head>\n";
    archivo << "<title>Practica 10</title>\n";
    archivo << "</head>\n";
    archivo << "<body>\n";

    archivo << "<h1>Practica 10 - Punteros al arreglo</h1>\n";

    archivo << "<h2>Numeros</h2>\n";
    archivo << "<ul>\n";

    for(int i=0; i<5; i++) {

        archivo << "<li>"
        << op.getDatos(i)
        << "</li>\n";
    }

    archivo << "</ul>\n";

    archivo << "<h2>Resultados</h2>\n";

    archivo << "<p>Suma: "
    << op.suma()
    << "</p>\n";

    archivo << "<p>Promedio: "
    << op.promedio()
    << "</p>\n";

    archivo << "<p>Media: "
    << op.media()
    << "</p>\n";

    archivo << "<p>Maximo: "
    << op.maximo()
    << "</p>\n";

    archivo << "<p>Minimo: "
    << op.minimo()
    << "</p>\n";

    archivo << "</body>\n";
    archivo << "</html>";

    archivo.close();
}

```

Para mantener un código limpio, modular y fácil de mantener, el proyecto se organizó rígidamente en tres archivos independientes:

- **Alumno.h (Archivo de cabecera):** Funciona como el "esqueleto" o índice de nuestra clase. Aquí se realiza la declaración de la clase, definiendo sus atributos, las firmas de sus métodos y estableciendo los niveles de acceso (público o privado) para proteger la información mediante encapsulamiento.
- **Alumno.cpp (Archivo de implementación):** Es el motor de la clase. Contiene el desarrollo lógico y la definición exacta del código de cada uno de los métodos declarados en la cabecera. Esta separación permite ocultar la complejidad interna del funcionamiento de la clase.
- **main.cpp (Archivo principal):** Es el punto de entrada para la ejecución del programa. Aquí se aloja la función main(), se manda a llamar (#include) a nuestra cabecera personalizada, se instancian los objetos, se define una estructura adicional y se ejecutan las instrucciones de salida a consola para visualizar los resultados.

A continuación, se muestran los errores que tuvimos a la hora de codificar.

Errores en main.cpp

Error 1: Sintaxis incorrecta de endl (Escritura mal visualizada) y separación

Anteriormente y por equivocación, se confundió una letra por un número, en esta parte, la l por el 1, la palabra correcta es "endl" (abreviatura de "end line"), esto sabiéndolo después por investigación del equipo a la hora del error en el compilador. El segundo fallo fue el espacio, separando el end del 1.

En la línea 8 del código original se observaba lo siguiente:

```
cout << "Tamaño de un entero" << sizeof(argc) << end 1<< end1<< end1;
```

Error 2: Declaración incorrecta de estructura

La estructura definida en las líneas 5-8 presentaba una sintaxis incorrecta:

```
struct{  
    char x[90], b[10];  
    double c;  
};AlumnoS;
```

El punto y coma después de la llave de cierre es incorrecto, además de que "AlumnoS" aparecía como una variable global mal declarada.

Error 3: Sensibilidad a mayúsculas y minúsculas (Declaración de objeto no reconocida)

Por descuido al teclear rápido, el equipo escribió el nombre de la clase Alumno con minúscula inicial al intentar crear el objeto a. Debido a que C++ es un lenguaje estrictamente sensible a mayúsculas y minúsculas (case-sensitive), el compilador no reconocía el tipo de dato alumno y arrojaba un error indicando que el identificador no estaba declarado.

En la línea 13 se veía lo siguiente:

```
13 | alumno a = alumno();
```

Errores en Alumno.h

Error 4: Declaración incorrecta de métodos sobrecargados

Se declararon dos métodos idénticos sin diferenciación de parámetros, además de que la sobrecarga de funciones requiere diferentes firmas esto quiere decir de diferentes parámetros, el compilador no puede distinguir entre ambas declaraciones y por ende marca error. Originalmente se pretendía tener un método sin parámetros y otro con un parámetro entero, se visualizaba de la siguiente manera:

```
void msg();  
void msg();
```

Errores en Alumno.cpp

Error 5: Falta de inclusión del header

El archivo de implementación comenzaba directamente con:

```
#include <iostream>  
using namespace std;
```

Este error es crítico porque el compilador no conoce la definición de la clase Alumno, las implementaciones de los métodos no pueden asociarse con la clase y genera errores de "undefined reference" durante la vinculación.

Proceso de corrección

Corrección en main.cpp

Solución para el error 1: Sintaxis incorrecta de endl (Escritura mal visualizada) y separación

Después de la investigación, se reemplazaron todas las instancias incorrectas de "end1" y "end 1" por "endl", ahora aparece de la siguiente manera:

```
cout << "Tamaño de un entero: " << sizeof(argc) << endl << endl << endl;
```

Solución para el error 2: Declaración incorrecta de estructura

Se eliminó el ; después de la llave al leer detenidamente lo que marcaba que el consolador, dando por terminado el error y sin que "AlumnoS" no apareciera como una variable global mal declarada. Ahora podemos ver que está así:

```
struct{  
    char x[90], b[10];  
    double c;  
} AlumnoS ;
```

Solución para el error 3: Sensibilidad a mayúsculas y minúsculas (Declaración de objeto no reconocida)

Después de una exhausta revisión línea por línea y de ser encontrado el detalle, la solución al fin fue corregir la sintaxis para que coincidiera exactamente con el nombre de la clase importada desde la cabecera "Alumno.h":

```
Alumno a = Alumno();
```

Corrección en Alumno.h

Solución para el error 4: Declaración incorrecta de métodos sobrecargados

Se corrigió la declaración de los métodos para que pudieran tener diferentes firmas y que el compilador pudiera diferenciar uno del otro, quedando así:

```
void msg();  
void msg(int a);
```

Corrección en Alumno.cpp

Solución del error 5: Falta de inclusión del header

Se agregó la inclusión del archivo de cabecera al inicio, también se verificó que las implementaciones coincidieran con las nuevas declaraciones, viéndose finalmente de la siguiente forma:

```
#include "Alumno.h"  
#include <iostream>
```

```
void Alumno::msg() {  
    cout << "salida vacia" << endl;  
}  
  
void Alumno::msg(int a) {  
    cout << "salida valor: " << a << endl;  
}
```

La siguiente página del documento, mostrará los códigos completos, corregidos y compilados de manera exitosa, así como a su vez, las salidas que se obtuvieron.

Errores en el compilador / entorno de desarrollo:

Error: Referencia indefinida a métodos de la clase (Falta de compilación múltiple)

Al momento de querer correr el programa final en Visual Studio Code, nos encontramos con un error en la terminal. Al ejecutar el comando para compilar, la consola nos arrojó varios mensajes de *"undefined reference"* (referencia indefinida) que apuntaban a las funciones de nuestra clase Alumno.

Analizando el problema, nos dimos cuenta de que el comando que se estaba ejecutando solo tomaba en cuenta el archivo main.cpp. Aunque el main sí detectaba que la clase existía (gracias a que le pusimos `#include "Alumno.h"`), el compilador no encontraba el archivo Alumno.cpp donde realmente estaba escrito el código de las funciones. Al faltar ese archivo, marcaba error porque no sabía cómo ejecutar los métodos.

```
Executing task: C:\msys64\ucrt64\bin\g++.EXE -Wall -Wextra -g3 e:\Documents\Escuela\Universidad\Tercer Semestre\Abstraccion\Primer Parcial\Alumno\main.cpp -o e:\Documents\Escuela\Universidad\Tercer Semestre\Abstraccion\Primer Parcial\Alumno\output\main.exe

e:\Documents\Escuela\Universidad\Tercer Semestre\Abstraccion\Primer Parcial\Alumno\main.cpp: In function 'int main(int, char**)':
e:\Documents\Escuela\Universidad\Tercer Semestre\Abstraccion\Primer Parcial\Alumno\main.cpp:11:27: warning: unused parameter 'argv' [-Wunused-parameter]
   11 | int main(int argc, char** argv) {
       |                      ~~~~~~^
C:/msys64/ucrt64/bin/./lib/gcc/x86_64-w64-mingw32/15.2.0/./././././x86_64-w64-mingw32/bin/ld.exe: C:\Users\USER\AppData\Local\Temp\ccmAZHw1.o: in function `main':
E:/Documents/Escuela/Universidad/Tercer Semestre/Abstraccion/Primer Parcial/Alumno/main.cpp:13:(.text+0x78): undefined reference to `Alumno::Alumno()'
C:/msys64/ucrt64/bin/./lib/gcc/x86_64-w64-mingw32/15.2.0/./././././x86_64-w64-mingw32/bin/ld.exe: E:/Documents/Escuela/Universidad/Tercer Semestre/Abstraccion/Primer Parcial/Alumno/main.cpp:17:(.text+0x136): undefined reference to `Alumno::msg()'
C:/msys64/ucrt64/bin/./lib/gcc/x86_64-w64-mingw32/15.2.0/./././././x86_64-w64-mingw32/bin/ld.exe: E:/Documents/Escuela/Universidad/Tercer Semestre/Abstraccion/Primer Parcial/Alumno/main.cpp:19:(.text+0x147): undefined reference to `Alumno::~Alumno()'
C:/msys64/ucrt64/bin/./lib/gcc/x86_64-w64-mingw32/15.2.0/./././././x86_64-w64-mingw32/bin/ld.exe: E:/Documents/Escuela/Universidad/Tercer Semestre/Abstraccion/Primer Parcial/Alumno/main.cpp:19:(.text+0x15a): undefined reference to `Alumno::~Alumno()'
collect2.exe: error: ld returned 1 exit status

The terminal process "C:\msys64\ucrt64\bin\g++.EXE -Wall, -Wextra, -g3, 'e:\Documents\Escuela\Universidad\Tercer Semestre\Abstraccion\Primer Parcial\Alumno\main.cpp', '-o', 'e:\Documents\Escuela\Universidad\Tercer Semestre\Abstraccion\Primer Parcial\Alumno\output\main.exe'" terminated with exit code: 1.
Terminal will be reused by tasks, press any key to close it.
```

Solución al Error de vinculación (Compilación de múltiples archivos fuente)

Para solucionar el conflicto de referencias indefinidas, se determinó que era estrictamente necesario indicarle al compilador que procesara y enlazara todos los archivos fuente (.cpp) involucrados en el proyecto de forma simultánea. Se abandonó la tarea de compilación automática que solo tomaba un archivo y se ejecutó manualmente el comando completo en la terminal:

```
g++ main.cpp Alumno.cpp -o main.exe
./main.exe
```

Al incluir explícitamente ambos archivos fuente, el compilador logró vincular exitosamente las llamadas a los objetos en el archivo principal con la lógica desarrollada en la implementación de la clase, generando el ejecutable sin contratiempos.

1. main.cpp (VERSIÓN FINAL)

```
1  #include "Alumno.h"
2  #include <iostream>
3
4  using namespace std;
5
6  struct {
7      char x[90], b[10];
8      double c;
9  } AlumnoS;
10
11 int main(int argc, char** argv) {
12     cout << "tamno de un entero" << sizeof(argc) << endl << endl << endl;
13     Alumno a = Alumno();
14     cout << "tamno de un entero -- " << sizeof(a) << endl << endl << endl;
15     cout << "tamno de un entero -- " << sizeof(AlumnoS) << endl << endl << endl;
16
17     a.msg();
18     return 0;
19 }
```

2 Alumno.h (VERSIÓN FINAL)

```
1  #ifndef ALUMNO_H
2  #define ALUMNO_H
3
4  class Alumno {
5  public:
6      Alumno();
7      ~Alumno();
8      void msg();
9      void msg(int a);
10 };
11
12 #endif
```

3. Alumno.cpp (VERSIÓN FINAL)

```
1  | #include "Alumno.h"
2  | #include <iostream>
3
4  | using namespace std;
5
6  | Alumno::Alumno() {
7  | }
8
9  | Alumno::~~Alumno() {
10 | }
11
12 | void Alumno::msg() {
13 |     cout << "salida vacia" << endl;
14 | }
15
16 | void Alumno::msg(int a) {
17 |     cout << "salida valor" << a << endl;
18 | }
```

PROGRAMA EJECUTADO:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS E:\Documents\Escuela\Universidad\Tercer Semestre\Abstraccion\Primer Parcial\Alumno> ./main.exe
tamno de un entero4

tamno de un entero -- 1

tamno de un entero -- 112

salida vacia
○ PS E:\Documents\Escuela\Universidad\Tercer Semestre\Abstraccion\Primer Parcial\Alumno> |
```

Archivos generados con base a la práctica anterior:

Para la realización de este programa se usó como base el que previamente se había realizado, solo que implementamos más tipos de datos y también se usaron structs para poder simular el concepto de “herencia”. Este programa sirve para medir el tamaño (en bytes) que ocupan distintos tipos de datos en memoria, usando el operador *sizeof* de C++.

Se trabajó con:

- **Tipos primitivos:** char, short, int, long, float, double, bool
- **Arreglos:** char[], int[], string[]
- **Matrices:** int[4][4] y int[4][3][2]
- **Structs:** Estructuras (NombreCompleto, PersonaE y DatoS)
- **Arreglo de structs:** PersonaE sec3Cm30[45] para representar un grupo (45 personas)

Cada archivo se mantuvo con el mismo rol dentro del programa, aunque con algunas modificaciones.

En el nuevo código se agregó lo siguiente:

1. Los structs NombreCompleto y PersonaE
- NombreCompleto guarda ap, am, ns como arreglos de char.
 - PersonaE contiene un NombreCompleto y además las variables de género y edad.


```

struct NombreCompleto{
    char ap [30];
    char am [30];
    char ns[30];
};

struct PersonaE{
    NombreCompleto nC;
    char genero;
    int edad;
};

```

2. Más tipos de datos dentro del struct DatoS, incluyendo:

- arreglos int[6]
- string[2]
- matriz int[4][4]
- matriz 3D int[4][3][2]

```

27 struct{
28     char x[90], b[10];
29     double c;
30     short bShort;
31     long d;
32     float f;
33     bool op;
34     string srt;
35     int lista_num[6] = {7,4,7,3,6,5}; //array de numeros
36     string carros[2]= {"Corvette","Audi"}; //array de objetos
37     char o;
38     int MxA [4][4];
39     int MxE [4][3][2];
40 }DatoS;

```

3. Se aplicó sizeof a todas las nuevas variables y estructuras:

- NombreCompleto

- PersonaE

- sec3Cm30[45]

```
cout << "Tamano de NombreCompleto: " << sizeof(NombreCompleto) <<endl <<endl;
cout << "Tamano de PersonaE: " << sizeof(PersonaE) <<endl <<endl;
cout << "Tamano de PersonaE[45]: " << sizeof(sec3Cm30) <<endl <<endl;
```

```
a.msg();|
return 0;
```

4. Se creó una variable, en donde se almacenarán los atributos que contenga la persona 1; denominada dentro del programa como “p1”::

- PersonaE p1;
y le asigné valores.

```
//Se debe de declarar dentro del main
//Se crea una variable del tipo "PersonaE"
PersonaE p1;
```

```
//strcpy convierte el char en string
strcpy(p1.nC.ap, "Perez"); //copia la cadena "Perez" dentro del arreglo char ap
strcpy(p1.nC.am, "Lopez");
strcpy(p1.nC.ns, "Juan");
```

```
p1.genero = 'M';
p1.edad = 20;
```

```
cout << p1.nC.ns << " "
    << p1.nC.ap << " "
    << p1.nC.am << "\n"
    << "Genero: " << p1.genero << "\n"
    << "Edad: " << p1.edad << endl;
```

```
PersonaE sec3Cm30[45];
```

También, nos dimos cuenta que para poder asignar valor a los campos de tipo char[] no se usa “=”, sino “strcpy”.

De igual manera, se necesitaron declarar nuevas librerías para el correcto funcionamiento del código:

```
1  #include "Datos.h"
2  #include <cstring>
3  #include <iostream>
4  #include <string>
5
```

Errores en mainDatos.cpp:

Error 1. Declaración incorrecta del struct NombreCompleto

Durante la compilación del programa con el comando:

```
g++ .cpp -o programa
```

El compilador arrojó múltiples errores, principalmente en la sección donde se declararon los struct.

El error principal fue:

```
-----
: error: 'NombreCompleto' does not name a type
```

Este error indica que NombreCompleto no es reconocido como un tipo de dato válido.

Este mismo error ocurrió debido a que el struct fue declarado de forma incorrecta.

```
struct {
    char ap [30], am [30], ns[30];
}NombreCompleto;
```

Esta declaración no crea un nuevo tipo llamado NombreCompleto.

Lo que hace es:

- Crear una variable global llamada NombreCompleto

Por lo tanto, cuando posteriormente se intentó usar:

```

struct {
    NombreCompleto nC;
    char genero;
    int edad;
}PersonaE;

```

El compilador no lo reconoció como tipo, ya que solo existía como variable global.

Debido a este error, el compilador generó errores adicionales:

```

|
|
|
mainDatos.cpp: In function 'int main(int, char**)':
mainDatos.cpp:79:13: error: expected ';' before 'p1'
79 |     PersonaE p1;
    |                ^~~
    |                ;
mainDatos.cpp:82:12: error: 'p1' was not declared in this scope
82 |     strcpy(p1.nC.ap, "Perez"); //copia la cadena "Perez" dentro del arreglo char ap
    |            ^~
mainDatos.cpp:95:13: error: expected ';' before 'sec3Cm30'
95 |     PersonaE sec3Cm30[45];
    |                ^~~~~~
    |                ;
mainDatos.cpp:99:50: error: 'sec3Cm30' was not declared in this scope
99 |     cout << "Tamano de PersonaE[45]: " << sizeof(sec3Cm30) <<endl <<endl;
    |                                                    ^~~~~~

```

Error 2. Declaración incorrecta de PersonaE:

Después de corregir el error relacionado con NombreCompleto, surgió un nuevo error durante la compilación:

```

mainDatos.cpp: In function 'int main(int, char**)':
mainDatos.cpp:79:13: error: expected ';' before 'p1'
   79 |     PersonaE p1;
      |                ^~
      |                ;
mainDatos.cpp:82:12: error: 'p1' was not declared in this scope
   82 |     strcpy(p1.nC.ap, "Perez"); //copia la cadena "Perez" dentro del arreglo char ap
      |            ^~
mainDatos.cpp:95:13: error: expected ';' before 'sec3Cm30'
   95 |     PersonaE sec3Cm30[45];
      |                ^~~~~~
      |                ;
mainDatos.cpp:99:50: error: 'sec3Cm30' was not declared in this scope
   99 |     cout << "Tamano de PersonaE[45]: " << sizeof(sec3Cm30) <<endl <<endl;
      |                                                  ^~~~~~

```

El problema fue que el struct PersonaE estaba declarado de manera incorrecta:

```

struct {
    NombreCompleto nC;
    char genero;
    int edad;
}PersonaE;

```

Esta declaración no crea un tipo llamado PersonaE.

En realidad, lo que hace es:

- Crear una variable global llamada PersonaE.

Por lo tanto, PersonaE no existía como tipo de dato, sino únicamente como una variable global.

Cuando posteriormente se intentó declarar:

```

79     PersonaE p1;
80     PersonaE sec3Cm30[45];

```

El compilador no reconoció PersonaE como tipo válido, generando errores de sintaxis.

Debido a que PersonaE no fue reconocido como tipo:

- No se pudo declarar la variable p1.

- Tampoco se pudo declarar el arreglo sec3Cm30.
- Las líneas donde se utilizaba p1 produjeron errores adicionales como:

```
,  
error: 'p1' was not declared in this scope
```

SOLUCIÓN CORRECTA

Solución a Error 1).

Este struct define un nuevo tipo de dato que representa un nombre completo mediante arreglos de caracteres.

Cada arreglo tiene un tamaño fijo de 30 bytes.

```
struct NombreCompleto{  
    char ap [30];  
    char am [30];  
    char ns[30];  
};
```

Y, posteriormente se define la struct “PersonaE” de esta forma:

```
struct PersonaE{  
    NombreCompleto nC;  
    char genero;  
    int edad;  
};
```

Si se quiere crear alguna variable con este struct, se realiza de la siguiente forma:

Este struct contiene otro struct dentro (composición).

Se demuestra cómo un tipo puede contener otro tipo previamente definido.

Solución a Error 2).

La solución fue declarar correctamente el struct como tipo:

```
17 struct PersonaE{
18     NombreCompleto nC;
19     char genero;
20     int edad;
21 };
```

Con esta corrección:

- PersonaE se reconoce como un tipo válido.
- Se pueden declarar variables como p1.
- Se pueden crear arreglos como PersonaE sec3Cm30[45];
- Todos los errores desaparecen.

1. mainDatos.cpp (VERSIÓN FINAL)

```

1  #include "Datos.h"
2  #include <cstring>
3  #include <iostream>
4  #include <string>
5
6  using namespace std;
7
8  struct NombreCompleto{
9      char ap [30], am [30], ns[30];
10 };
11
12 struct PersonaE{
13     NombreCompleto nC;
14     char genero;
15     int edad;
16 };
17
18
19
20 struct{
21     char x[90], b[10];
22     double c;
23     short bShort;
24     long d;
25     float f;
26     bool op;
27     string srt;
28     int lista_num[6] ={7,4,7,3,6,5}; //array de numeros
29     string carros[2]= {"Corvette","Audi"}; //array de objetos
30     char o;
31     int MxA [4][4];
32     int MxE [4][3][2];
33 }DatoS;
34
35
36 int main(int argc, char** argv){
37
38     cout << "tamano de un entero -- " << sizeof(argc) << endl << endl << endl;
39
40     Datos a = Datos();
41
42     cout << "tamano del objeto Datos -- " << sizeof(a) << endl << endl << endl;
43
44     cout << "tamano de un struct DatoS-- " << sizeof(DatoS) << endl << endl << endl;
45
46     cout << "tamano del arreglo char x[90] -- " << sizeof(DatoS.x) << "\n\n";
47
48     cout << "tamano del arreglo char b[10] -- " << sizeof(DatoS.b) << "\n\n";
49
50     cout << "tamano del double -- " << sizeof(DatoS.c) <<endl <<endl <<endl;

```



```

52     cout << "tamano del short -- " << sizeof(DatoS.bShort) << endl << endl << endl;
53
54     cout << "tamano del long -- " << sizeof(DatoS.d) << endl << endl << endl;
55
56     cout << "tamano del float -- " << sizeof(DatoS.f) << endl << endl << endl;
57
58     cout << "tamano del booleano -- " << sizeof(DatoS.op) << endl << endl << endl;
59
60     cout << "tamano de char -- " << sizeof(DatoS.o) << endl << endl << endl;
61
62     cout << "tamano de un string -- " << sizeof(DatoS.srt) << endl << endl << endl;
63
64     cout << "tamano de un arreglo int[6] -- " << sizeof(DatoS.lista_num) << endl << endl << endl;
65
66     cout << "tamano de un arreglo string[2] -- " << sizeof(DatoS.carros) << endl << endl << endl;
67
68     cout << "tamano de matriz int[4][4] -- " << sizeof(DatoS.MxA) << endl << endl << endl;
69
70     cout << "tamano de matriz int[4][3][2] -- " << sizeof(DatoS.MxE) << endl << endl << endl;
71
72     //Se debe de declarar dentro del main
73     //Se crea una variable del tipo "PersonaE"
74     PersonaE p1;
75     PersonaE sec3Cm30[45];
76
77     //strcpy convierte el char en string
78     strcpy(p1.nC.ap, "Perez"); //copia la cadena "Perez" dentro del arreglo char ap
79     strcpy(p1.nC.am, "Lopez");
80     strcpy(p1.nC.ns, "Juan");
81
82     p1.genero = 'M';
83     p1.edad = 20;
84
85     cout << p1.nC.ns << " "
86         << p1.nC.ap << " "
87         << p1.nC.am << "\n"
88         << "Genero: " << p1.genero << "\n"
89         << "Edad: " << p1.edad << endl;
90
91
92
93     cout << "Tamano de NombreCompleto: " << sizeof(NombreCompleto) << endl << endl;
94     cout << "Tamano de PersonaE: " << sizeof(PersonaE) << endl << endl;
95     cout << "Tamano de PersonaE[45]: " << sizeof(sec3Cm30) << endl << endl;
96
97     a.msg();
98     return 0;
99
100 }

```

2. Datos.h (VERSIÓN FINAL)

```
1  #ifndef DATOS_H
2  #define DATOS_H
3  #include <iostream>
4
5  class Datos {
6  public:
7      Datos();
8      ~Datos();
9
10     void msg();
11     void msg(int a);
12 };
13
14 #endif
```

3. Datos.cpp (VERSIÓN FINAL)

```
1  #include "Datos.h"
2  #include <iostream>
3
4  using namespace std;
5
6  Datos::Datos() {}
7  }
8
9  Datos::~Datos() {
10 }
11
12 void Datos::msg() {
13     cout << "salida vacia" << endl;
14 }
15
16 void Datos::msg(int a) {
17     cout << "salida valor" << a << endl;
18 }
19
20
```

PROGRAMA EJECUTADO:

```
PS C:\Users\drakc\sizeofdedatos> g++ *.cpp -o programa
```

```
PS C:\Users\drakc\sizeofdedatos> .\programa
```

```
tamano de un entero -- 4
```

```
tamano del objeto Datos -- 1
```

```
tamano de un struct Datos-- 416
```

```
tamano del arreglo char x[90] -- 90
```

```
tamano del arreglo char b[10] -- 10
```

```
tamano del double -- 8
```

```
tamano del short -- 2
```

```
tamano del long -- 4
```

```
tamano del float -- 4
```

```
tamano del booleano -- 1
```

```
tamano de char -- 1
```

```
tamano de un string -- 32
```

```
tamano de un arreglo int[6] -- 24
```

```
tamano de un arreglo string[2] -- 64
```

```
tamano de matriz int[4][4] -- 64
```

```
tamano de matriz int[4][3][2] -- 96
```

Juan Perez Lopez

Genero: M

Edad: 20

Tamano de NombreCompleto: 90

Tamano de PersonaE: 96

Tamano de PersonaE[45]: 4320

salida vacia

□

A través de los seis archivos trabajados (los tres originales del profesor y los tres ampliados en la tarea), se logró comprender cómo se representan y almacenan los distintos tipos de datos en memoria en C++.

En la **primera versión** se analizaron conceptos básicos como la organización del programa en archivos `.h`, `.cpp` y `main`, el uso de clases, métodos y el operador `sizeof` para medir el tamaño en bytes de variables, objetos y estructuras.

En la **segunda versión** se ampliaron los tipos de datos, incorporando arreglos, matrices bidimensionales y tridimensionales, así como estructuras anidadas (`NombreCompleto` y `PersonaE`) y arreglos de estructuras (`PersonaE sec3Cm30[45]`). Esto permitió observar cómo el tamaño en memoria aumenta según la complejidad y cantidad de datos almacenados.

Además, los errores presentados durante la compilación ayudaron a reforzar conceptos importantes como la diferencia entre tipo y variable, la correcta declaración de `struct` y la importancia de la sintaxis adecuada en C++.

En conclusión, el ejercicio permitió comprender tanto la estructura de un programa en C++ como la representación y almacenamiento de la información en memoria.

Después de desarrollar los seis archivos del trabajo – los tres base que nos proporcionó el profesor, más los tres que ampliamos para la tarea – pude entender de forma práctica cómo se representan y guardan los distintos tipos de datos en memoria cuando se programa en C++.

En la primera versión se trabajó con la estructura básica del programa, entendiendo cómo se divide en distintos archivos y cómo se conectan entre sí. También se utilizó el operador `sizeof` para observar cuánto espacio ocupan en memoria algunas variables y estructuras.

Luego, en la versión ampliada, agregué nuevos tipos de datos primitivos, arreglos de una sola dimensión, matrices bidimensionales y tridimensionales, así como estructuras anidadas (como `NombreCompleto` dentro de `PersonaE`). También hice un arreglo de estas estructuras (`PersonaE sec3Cm30[45]`), lo que me permitió ver claramente cómo aumenta el consumo de memoria cuando se multiplica un tipo de dato definido por el usuario.

Durante todo el proceso se presentaron varios errores que ayudaron mucho a reforzar ideas clave: la diferencia entre tipo y variable, por qué importa el orden en que declaramos las cosas y cuál es la estructura sintáctica que espera el lenguaje.

Al final, el ejercicio no solo sirvió para medir el tamaño en bytes de cada tipo de dato, sino también para entender cómo se organizan en memoria, cómo se construyen estructuras complejas a partir de tipos simples y cómo el compilador interpreta cada una de nuestras declaraciones.